TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 10

Git config, first commits, status, diff & log

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. Module 04 — Git and GitHub: overview
2. 04.00 Prerequisites Check
3. 04.01 Configure Git
4. 04.02 Init and First Commit
5. 04.03 The status / add / commit Cycle
6. 04.04 Diff and Log

Module 04 — Git and GitHub

What you'll be able to do after this module

- Configure Git globally with `user.name`, `user.email`, and `init.defaultBranch` set to `main`
- Initialize a repository and manage changes through the status / add / commit cycle with descriptive commit messages
- Inspect changes and history confidently using `git status`, `git diff`, and `git log` with their most useful flags
- Create, switch, merge, and delete branches, distinguishing fast-forward merges from true merge commits
- Recover from common mistakes using `git restore`, `git restore --staged`, `git commit --amend`, and `git reset`
- Generate an ed25519 SSH key, register it with a GitHub account, and push a local repository to a GitHub remote
- Clone a repository from GitHub, pull updates, and resolve a merge conflict by hand
- Open a pull request on GitHub, review and respond to inline comments, and merge it back to `main`

Lessons and objectives

#	Lesson	Learning objective
00	Prerequisites Check	Verify that Git is installed and the Module 03 FastAPI project exists with no pre-existing `.git` directory.
01	Configure Git	Configure Git globally with `user.name`, `user.email`, `init.defaultBranch=main`, and `pull.rebase=false`, then verify the settings stored in `~/.gitconfig`.
02	Init and First Commit	Initialize a Git repository in the FastAPI project, extend `.gitignore`, stage everything with `git add .`, and make a first commit on `main`.
03	The status / add / commit Cycle	Make a series of commits using the edit, `git status`, `git add`, `git commit` cycle, including selective staging and the `-am` shortcut's untracked-file limitation.
04	Diff and Log	Inspect unstaged, staged, and between-commit changes with `git diff` and read history with `git log` filters such as `--oneline`, `--graph`, `--author`, and `--grep`.
05	Branches	Create and switch branches with `git switch`, merge a feature branch into `main` via both a fast-forward and a true merge commit, and delete merged branches with `git branch -d`.
06	Undoing Things	Undo common mistakes by discarding edits with `git restore`, unstaging with `git restore --staged`, amending the last commit with `git commit --amend`, and removing a commit with `git reset`.
07	GitHub Account and SSH Key	Generate an ed25519 SSH key with `ssh-keygen`, register the public key in GitHub's settings, and verify passwordless authentication with `ssh -T`.
08	Pushing to GitHub	Connect a local repository to a GitHub remote with `git remote add origin`, push it with `git push -u origin main`, and round-trip commits between the terminal and the web UI with `git push` and `git pull`.
09	Cloning, Pulling, and Conflicts	Clone a repository from GitHub, sync changes between two working copies with `git pull`, and resolve a merge conflict by editing out the conflict markers, staging the file, and committing the merge.
10	Pull Requests	Push a feature branch to GitHub, open a pull request, leave and respond to inline review comments, merge the PR with a merge commit, and sync the result to the local `main`.
11	Final Checkpoint	Complete a full branch, commit, push, pull request, review, and merge cycle from memory and verify the new `/info` route works end to end.

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

Learning objective: Verify that Git is installed and the Module 03 FastAPI project exists with no pre-existing `.git` directory.

Lesson 00: Prerequisites Check

By the end, you will have confirmed Git is installed and your Module 03 FastAPI project still exists.

~5 minutes

---

1. Open VS Code and connect to your Ubuntu environment. Open the integrated terminal.

2. Confirm Git is installed (Module 01 lesson 5 installed it).

```
git --version
```

```
git version 2.34.1
```

(Any 2.x is fine.)

3. Confirm your FastAPI project from Module 03 is still there.

```
ls ~/docker-course/fastapi-app/main.py
```

You should see the path printed. If not, you skipped Module 03 — go back.

4. Confirm there's no existing `.git` directory in the project (we want to start clean).

```
ls -la ~/docker-course/fastapi-app/.git 2>&1 | head -n 1
```

You should see `ls: cannot access ...` — meaning no repo yet. **If a `.git` directory already exists, raise your hand** before continuing. We need to know what's there.

5. Confirm `~/docker-course/notes/notes.txt` exists.

```
ls ~/docker-course/notes/notes.txt
```

6. Log this check.

```
echo "Module 04 prereq check: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

Watch out

- If `git --version` fails, run `sudo apt install -y git`.
- If `~/docker-course/fastapi-app/` is missing or empty, you'll have to redo Module 03 first. There's no shortcut.

Checkpoint

```
git --version
```

Should print a version.

```
ls ~/docker-course/fastapi-app/
```

Should list `main.py`, `routers/`, `requirements.txt`, etc.

Next up: 01-git-config.md — Tell Git who you are

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Configure Git globally with `user.name`, `user.email`, `init.defaultBranch=main`, and `pull.rebase=false`, then verify the settings stored in `~/.gitconfig`.

Lesson 01: Configure Git

By the end, Git knows your name, your email, and that new repos should default to a `main` branch.

~10 minutes of typing

> Git stamps every commit with a name and email. If you don't set them up front, your first commit fails with a confusing error. Set them once, globally, and never think about it again.

---

1. Set your name (this shows up on every commit).

```
git config --global user.name "Your Name"
```

(Replace `Your Name` with your actual name. Quotes matter if it has a space.)

2. Set your email. **Use the same email you'll use for your GitHub account in lesson 07.**

```
git config --global user.email "you@example.com"
```

3. Set the default branch name to `main` (some older Gits default to `master`).

```
git config --global init.defaultBranch main
```

4. Make Git's output a bit more readable.

```
git config --global color.ui auto
```

5. Configure how merges handle pull conflicts. The safe default for beginners is "fail unless I say otherwise" — set it explicitly.

```
git config --global pull.rebase false
```

6. Read back what you set.

```
git config --global --list
```

You should see something like:

```
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
color.ui=auto
pull.rebase=false
```

7. Read just one value at a time — this pattern is useful when scripts need to know your config.

```
git config --global user.name
```

```
Your Name
```

8. See where Git stores your global config.

```
ls -la ~/.gitconfig
```

You should see the file. It's plain text — you could edit it by hand, but `git config` is safer.

9. Take a peek inside.

```
cat ~/.gitconfig
```

```
[user]
    name = Your Name
    email = you@example.com
[init]
    defaultBranch = main
[color]
    ui = auto
[pull]
    rebase = false
```

10. Log it.

```
echo "Git configured: $(git config --global user.name) <$(git config --global user.email)>" >> ~/docker-course/notes/notes.txt
```

Watch out

- **Use a real email** that matches what you'll use on GitHub. Mismatched emails create commits that don't link to your GitHub profile.
- `--global` writes to `~/.gitconfig` and applies to every repo for your user. Without `--global`, the setting only applies to the current repo.
- If you typo your name or email, just re-run the command — the new value overwrites the old.

Checkpoint

```
git config --global user.name && git config --global user.email
```

Should print your name and email on two lines.

```
git config --global init.defaultBranch
```

Should print `main`.

```
test -f ~/.gitconfig && echo "OK"
```

Should print `OK`.

Next up: 02-init-and-first-commit.md — Turn the FastAPI project into a Git repo

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Initialize a Git repository in the FastAPI project, extend `.gitignore`, stage everything with `git add .`, and make a first commit on `main`.

Lesson 02: Init and First Commit

By the end, your FastAPI project is a Git repository with one commit on `main`.

~15 minutes of typing

---

1. Move into the FastAPI project.

```
cd ~/docker-course/fastapi-app
```

2. Confirm where you are.

```
pwd
```

```
/home/your-user/docker-course/fastapi-app
```

3. Initialize a new Git repository here.

```
git init
```

```
Initialized empty Git repository in /home/.../fastapi-app/.git/
hint: Using 'master' as the name of the initial branch...
```

(If you see a hint about `master`, you missed step 3 of lesson 01 — fix it now and re-run `git init`.)

4. See the hidden `.git` directory that just appeared.

```
ls -la .git | head -n 10
```

You should see `HEAD`, `config`, `objects/`, `refs/`. **Do not edit anything inside `.git/` by hand.** All `git` commands manipulate this folder for you.

5. Confirm the current branch is `main`.

```
git branch
```

(No output yet — you have no commits, so the branch doesn't fully exist yet. We'll see it after the first commit.)

6. See the status of the new repo.

```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        .env
        .gitignore
        Dockerfile
        database.py
        ...
```

Read this carefully. Git is telling you everything is "untracked" — it sees the files but isn't versioning them yet.

7. Look at your existing `.gitignore` (you wrote it in Module 03 lesson 01).

```
cat .gitignore
```

```
.venv/
.env
```

8. Confirm `.git` itself is in `.gitignore`? It shouldn't be — `.git` is the repo, not a tracked file. But let's add a few more sensible ignores.

```
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore
```

9. Confirm.

```
cat .gitignore
```

```
.venv/
.env
__pycache__/
*.pyc
```

10. Run status again — `.venv` and `.env` should now be ignored.

```
git status
```

The untracked-files list should no longer include `.venv/` or `.env`.

11. Stage everything for the first commit. The `.` means "everything in the current directory."

```
git add .
```

(No output = success.)

12. Run status again.

```
git status
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .dockerignore
        new file:   .gitignore
        new file:   Dockerfile
        ...
```

Everything moved from "Untracked" to "Changes to be committed" — the **staging area**.

13. Make the first commit. The `-m` flag attaches a message inline.

```
git commit -m "Initial commit: FastAPI app with Postgres and SQLAlchemy"
```

```
[main (root-commit) abc1234] Initial commit: FastAPI app with Postgres and
SQLAlchemy
 15 files changed, 200 insertions(+)
 create mode 100644 .dockerignore
 ...
```

Read the output. `[main (root-commit) abc1234]` — your first commit's short hash, on the `main` branch.

14. Status now.

```
git status
```

```
On branch main
nothing to commit, working tree clean
```

This is the calm Git state. Working tree clean = "everything is committed."

15. Look at the commit.

```
git log
```

```
commit abc1234... (HEAD -> main)
Author: Your Name <you@example.com>
Date:   ...

    Initial commit: FastAPI app with Postgres and SQLAlchemy
```

(Press `q` to exit if it pages.)

16. A more compact view.

```
git log --oneline
```

```
abc1234 (HEAD -> main) Initial commit: FastAPI app with Postgres and SQLAlchemy
```

17. See what's in the commit.

```
git show --stat HEAD
```

Lists every file in the commit and how many lines were added/removed.

18. Log it.

```
echo "Repo initialized + first commit: $(git log --oneline | head -n 1)" >>
~/docker-course/notes/notes.txt
```

Watch out

- **`git init` runs in the current directory** — make sure you're in `~/docker-course/fastapi-app` first, not your home directory. Initializing a repo in `~` is a classic accident.
- **Always `git status` between commands.** Git's state is invisible without it. Status is your single source of truth.
- The commit hash (`abc1234`) is unique to your machine — yours will look different. That's fine.
- A commit message is required. `-m "..."` provides it inline; without `-m`, Git opens an editor (set by `$EDITOR`) and you must save+quit.

Checkpoint

```
git status
```

Should say `nothing to commit, working tree clean`.

```
git log --oneline | wc -l
```

Should print `1`.

```
git log -1 --format="%an <%ae>"
```

Should print your name and email (matches what you set in lesson 01).

```
test -d .git && echo "OK: repo exists"
```

Should print `OK: repo exists`.

Next up: 03-status-add-commit.md — The day-to-day cycle of changing files

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Make a series of commits using the edit, `git status`, `git add`, `git commit` cycle, including selective staging and the `-am` shortcut's untracked-file limitation.

Lesson 03: The status / add / commit Cycle

By the end, you will have made several commits the way you'll make them every day for the rest of your career.

~15 minutes of typing

> The cycle: edit → `git status` → `git add` → `git commit -m "..."`. Repeat. That's 90% of Git in daily use.

---

1. You're in `~/docker-course/fastapi-app`. Working tree is clean from lesson 02.

```
cd ~/docker-course/fastapi-app
git status
```

Should say `nothing to commit, working tree clean`.

2. Make a tiny change. Open `main.py` in VS Code and change the hello-world response from `{"hello": "fastapi"}` to `{"hello": "git"}`. Save.

3. Run status.

```
git status
```

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.py

no changes added to commit (use "git add" to track)
```

Read it carefully. The file is modified but not staged.

4. See exactly what changed.

```
git diff
```

```
diff --git a/main.py b/main.py
index ...
--- a/main.py
+++ b/main.py
@@ -X,X +X,X @@ def read_root():
-    return {"hello": "fastapi"}
+    return {"hello": "git"}
```

Read it. Red lines (with `-`) are removed, green lines (with `+`) are added.

5. Stage just `main.py`.

```
git add main.py
```

6. Status again.

```
git status
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   main.py
```

The file moved from "Changes not staged" to "Changes to be committed" — the **staging area**.

7. Commit it.

```
git commit -m "Change hello response to git"
```

```
[main def5678] Change hello response to git
 1 file changed, 1 insertion(+), 1 deletion(-)
```

8. Status.

```
git status
```

Working tree clean again.

9. Log.

```
git log --oneline
```

You should see two commits.

10. Now make changes to **two** files and stage them selectively. Edit `main.py` again — change the response back to `{"hello": "fastapi"}` and save.

11. Edit `routers/items.py` — at the very top of the file (above the imports), add a single comment line:

```
# Items router — manages item CRUD
```

Save.

12. Status.

```
git status
```

You should see two modified files.

13. Stage only `main.py`.

```
git add main.py
```

14. Status.

```
git status
```

You should see `main.py` under "Changes to be committed" and `routers/items.py` still under "Changes not staged."

15. Commit just the staged change.

```
git commit -m "Restore hello response to fastapi"
```

16. Status.

```
git status
```

`routers/items.py` is still showing as modified — it wasn't part of the last commit.

17. Stage and commit it now.

```
git add routers/items.py
git commit -m "Add header comment to items router"
```

18. Look at history.

```
git log --oneline
```

You should see four commits now.

19. Try a "shortcut" you'll use often — `-am` stages and commits **all already-tracked, modified files** in one shot. Make a change first. Edit `routers/users.py` and add at the top:

```
# Users router — manages user CRUD
```

20. Use the shortcut.

```
git commit -am "Add header comment to users router"
```

```
[main ...] Add header comment to users router
 1 file changed, 1 insertion(+)
```

Note: `-am` only stages files Git already tracks. New (untracked) files still need a plain `git add`.

21. Demonstrate that. Create a new file.

```
echo "# Project notes" > NOTES.md
```

22. Try the shortcut.

```
git commit -am "Add notes file"
```

```
On branch main
Untracked files:
        NOTES.md

nothing added to commit but untracked files present
```

See? The new file wasn't picked up — `-am` ignores untracked files.

23. Stage it manually and commit.

```
git add NOTES.md
git commit -m "Add NOTES.md placeholder"
```

24. History check.

```
git log --oneline
```

You should now have around six commits.

25. Log it.

```
echo "Commit cycle drilled: $(git log --oneline | wc -l) commits total" >>
~/docker-course/notes/notes.txt
```

Watch out

- **`git status` between commands.** Make it a tic. It tells you exactly what state Git thinks you're in.
- **`git diff` shows unstaged changes.** To see staged-but-not-committed changes, use `git diff --staged`.
- **`-am` is a footgun for new files** — it silently skips them. Always check `git status` before committing to confirm what's about to be included.
- **Commit messages should be useful.** `"fix"` and `"wip"` are unhelpful. Write what changed and why. Imperative mood ("Add", "Fix", "Refactor") is the convention.

Checkpoint

```
git status
```

Should say `nothing to commit, working tree clean`.

```
git log --oneline | wc -l
```

Should print at least `5`.

```
git log -1 --format="%s"
```

Should print your most recent commit message.

Next up: 04-diff-and-log.md — Read history and diffs like a pro

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Inspect unstaged, staged, and between-commit changes with `git diff` and read history with `git log` filters such as `--oneline`, `--graph`, `--author`, and `--grep`.

Lesson 04: Diff and Log

By the end, you will be fluent in `git diff`, `git log`, and the most useful flags on each.

~15 minutes of typing

---

1. Confirm you're in the project and the tree is clean.

```
cd ~/docker-course/fastapi-app
git status
```

2. Look at the full log.

```
git log
```

(Press `q` to quit the pager.)

3. One-line per commit.

```
git log --oneline
```

4. With branch + tag decorations.

```
git log --oneline --decorate
```

You should see `(HEAD -> main)` next to the most recent commit.

5. With a tiny visual graph (more useful when you have branches).

```
git log --oneline --graph --decorate --all
```

6. Show file changes with each commit.

```
git log --stat --oneline
```

7. Show the actual diff with each commit.

```
git log -p
```

(Lots of output. Press `q` to quit.)

8. Filter to commits touching a specific file.

```
git log --oneline -- main.py
```

9. Filter by author.

```
git log --oneline --author="Your Name"
```

10. Filter by message text.

```
git log --oneline --grep="hello"
```

You should see your "Change hello response to git" and "Restore hello response to fastapi" commits.

11. Show just the last N commits.

```
git log --oneline -3
```

12. Look at one specific commit by hash. Pick any short hash from `git log --oneline`.

```
git show <paste-a-hash-here>
```

You'll see the commit metadata + the diff.

13. Now make changes to feel `git diff` properly. Edit `routers/items.py` and add a docstring under the `class ItemIn(BaseModel):` line:

```
    """Pydantic model for incoming item creation requests."""
```

14. Save. Diff.

```
git diff
```

You should see exactly the line you added.

15. See the diff with more context lines.

```
git diff -U10
```

(Default is `-U3` — three lines of context above and below each change.)

16. Stage the change.

```
git add routers/items.py
```

17. `git diff` now shows nothing — staged changes don't appear in plain `git diff`.

```
git diff
```

(No output.)

18. See the staged diff.

```
git diff --staged
```

(Same diff as before — but now you're looking at what's about to be committed.)

19. Commit it.

```
git commit -m "Add docstring to ItemIn model"
```

20. See the diff between the commit you just made and the commit before it.

```
git diff HEAD~1 HEAD
```

(`HEAD` = the latest commit. `HEAD~1` = one commit before HEAD. `HEAD~2` = two before, etc.)

21. Same diff in stat form.

```
git diff HEAD~1 HEAD --stat
```

22. See the full diff between any two commits — pick two short hashes from `git log --oneline`.

```
git diff <older-hash> <newer-hash>
```

23. Compare a single file across two commits.

```
git diff HEAD~3 HEAD -- main.py
```

24. A pretty log format you might want to alias later. Type it once now.

```
git log --pretty=format:"%h %an %ar  %s" -10
```

```
abc1234 Your Name 5 minutes ago  Add docstring to ItemIn model
def5678 Your Name 10 minutes ago  Add header comment to users router
...
```

25. Log it.

```
echo "Diff + log fluency drilled" >> ~/docker-course/notes/notes.txt
```

Watch out

- **`git diff` shows working-tree changes only.** For staged changes, `git diff --staged`. For both, `git diff HEAD`.
- **`HEAD~N`** counts back from your current commit. `HEAD~1` = parent, `HEAD~2` = grandparent. With merges this gets weirder; we won't worry about that yet.
- **Pager controls** (`less`): `q` to quit, `/text` to search, `n` for next match, space for next page, `b` for back.
- **Don't shell out to `cat` to see commits** — `git show` is the right tool.

Checkpoint

```
git log --oneline | wc -l
```

Should print at least `6`.

```
git log --oneline --grep="hello" | wc -l
```

Should print at least `1`.

```
git diff HEAD~1 HEAD --stat
```

Should show one file changed (the docstring you added).

Next up: 05-branches.md — Work on multiple things at once

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.